OCS INTELLIGENCE · SPRINT 2 · PLAYBOOK

The VR experience of the OCS LLM path

A Meta Quest experience that builds Rig 2, powers it on, and follows one request to the cards.

STATE **in progress** | DATE 2026-09-16 | PROJECT PROMX-OCS

SOURCE evidence/sprint2/2026-09-16-vr-experience.md

Sprint 2 built the VR layer of OCS Intelligence. In a Meta Quest headset, the viewer watches a GPU rig assemble itself, power on, and serve one inference request. The request travels from a student's desk through the system and lands on specific cards in front of them. The code is in `YashPatil321/VirtualLLMVisualizer`, with 42 commits from 2026-09-16 to 2026-10-12. The whole arc runs end to end in a headless simulation: 100 of 100 tests pass, and the arc takes 144 s. It has **not** yet been run on a headset, so frame rate and the outside-viewer test are still open.

At a glance

Goal	Five minutes in a headset that show what happens when a prompt is sent to our system
Outcome	Experience, scene builder, Quest setup and models complete. Full arc passes in simulation (2026-10-14). Not yet run on a Quest
Where	`VirtualLLMVisualizer`, commit `92544c4`. Unity 6000.6.1f1, URP, Android ARM64, IL2CPP, OpenXR
Why	Most people's picture of an LLM stops at a text box. This shows the rest of it

ACT	STARTS	WHAT THE VIEWER SEES
1. Empty bench	0.0 s	A bare two-level black frame and a tray of parts, in a dark data hall
2. Assembly	20.0 s	23 parts fly in on arcs, in Rig 2's real build order. Each part is labelled with its name and one fact as it seats
3. Power on	70.2 s	The cards power on in a ripple, fans spin up, and a shockwave crosses the floor and wakes the server racks
4. The request	85.2 s	Stations rise out of the floor around the rig. A pulse carries the prompt along a floor track, and each station's sign says what happened there and how long it took
5. The answer	124.2 s	GPUs 0–3 go to full load, tokens stream to the student's screen, and the answer types out. The answer then runs back along the track
End	144.2 s	The scene holds, and the viewer can walk between the stations. Total: 144 s against a 300 s budget

Before

At the start of Sprint 2, the VR layer was a design document and a hand-written trace format. There was no Unity project, no rig model and no scene, and nothing could be shown. The request path in the first draft also included a Mac Mini, which is not part of the system.

What we did

1. **Telemetry and data layers first** (2026-09-16 to 09-17). Plain C# classes load the trace, sort hops by time, validate them, and play them back as events. The data lives in ScriptableObjects, not code: the build order, the card layout, the stations and the narration. Playback logic never touches a Transform, so every rule can be tested without a scene.
2. **One-click scene builder** (2026-09-25). OCS > Build Experience Scene builds the whole scene from the data. It uses the real models when they are present, and placeholders otherwise.
3. **Rig 2 modelled as it really is** (2026-09-25 to 09-30). `tools/blender/generate_rig_parts.py` builds 8 models at real size, from photos of Rig 2: EVGA GTX 1070 SC cards, a two-level black frame and two Antec 1300 W supplies. The models are about 300 KB in total and are committed to the repo, so no Git LFS is needed.
4. **Removed the Mac Mini from the request path, and matched the VR to the week 1 architecture** (2026-09-30 to 10-01). The path is now: student → AWS relay (NGINX + TLS) → FastAPI broker with Redis and RDS → NetBird → llama-server on `rig-2`.
5. **Built the look within Quest 2 limits** (2026-09-30). There are no realtime shadows, no post-processing and no dynamic lights. All of the glow comes from three unlit, Single Pass Instanced shaders: `OCSGlow` for halos, beams, sparks and tokens, `OCSGridFloor` for the floor, the shockwave and the ripples, and `OCSRackLights` for a whole rack face of blinking lights on one quad.
6. **Made it informative** (2026-10-01). Every part has a fact line, and every station has a role. Two boards stand in the hall: "The rig at a glance" and "How a request flows". "How a request flows" lights each hop as the request reaches it, then shows the total time and the GPU's share. A timeline shows where the time goes.
7. **Quest setup** (2026-10-04 to 10-12). The project has a head-tracked camera, and an OCS > Quest menu that sets up URP, installs the XR packages, applies the build settings and checks the setup. The OpenXR loader, the Android build profile and the rebuilt `Experience.unity` were committed on 2026-10-12.
8. **Walking and controls** (2026-10-12). The left stick walks, the right stick turns and rises, and holding the right trigger runs time at 4×. The left menu button starts the experience over. In the editor the same controls are WASD, the arrow keys, F and R.

> **NOTE**
>
> The request shown is a hand-written trace of the **target** path from the week 1 deck, not a recording from the live API. The network hops are slowed down so the viewer can follow them; the GPU part plays in real time. The boards show the real durations from the trace.

How to check it

Without Unity (run on 2026-10-14 on commit `92544c4`, with Mono C# 6.8.0.105 and glslang 11:15.1.0):

```
git clone https://github.com/YashPatil321/VirtualLLMVisualizer
cd VirtualLLMVisualizer
sudo apt-get install -y mono-mcs glslang-tools
tools/headless-tests/run.sh
```

```
100 passed, 0 failed
AssemblySequence 23 steps, 50.2s   SystemGraph 7 nodes   NarrationTrack 14 lines
every key in every asset matched a real field.
assembly steps played : 23 of 23
total runtime         : 144s (budget 300s)
during the request    : GPUs 0-3 peak at 1.00, other cards at most 0.12 (idle glow)
beams                 : 6 of 6 lit as the request passed
journey board, end    : Total 10.02 s, the GPU took 98.1%
0 problem(s).
OCSGlow, OCSGridFloor, OCSRackLights: vert + frag, fog + no fog: all ok
```

This run shows that the code compiles, that every field in every `.asset` file matches a real C# field, and that the right cards light up and only those. It says nothing about frame rate, comfort, or whether the build runs on a Quest.

On a Quest (the full order is in `docs/demo-checklist.md`):

CHECK	PASS LOOKS LIKE	IF IT FAILS
OCS > Quest > 0. Set Up URP for Quest	"URP is active on every quality level"	Magenta everything: no URP asset active
Delete `Assets/Prefabs/Environment.prefab`, then OCS > Build Experience Scene	"23 parts from Assets/Art/Models"	An old room prefab replaced the data hall
Press Play	The arc runs in about 2.5 min. Nothing magenta, no mirrored text	Screenshot plus the Console
OCS > Quest > 1, 2, then Check Setup	Every line ok	Fix whatever line is red
XR Plug-in Management, Android	OpenXR, Meta Quest Support, Single Pass Instanced, Oculus Touch Controller Profile	No Touch profile: the sticks do nothing
Build And Run to a Quest 2 over USB	The app starts in the headset by itself	`adb devices` shows unauthorized: accept the USB debugging prompt in the headset
Meta performance overlay through power on and token streaming	72 fps throughout	Reduce in this order: fewer racks, less dust, then fog off
Three runs from a cold start	The same result each time	Record what changed

Pitfalls

SYMPTOM	CAUSE	FIX
Everything magenta except text	No URP pipeline asset active (`Assets/Settings/` missing or not assigned)	OCS > Quest > 0, or copy `Assets/Settings/` with its `.meta` files
Unity opens in Safe Mode, and editor commands fail with nothing useful	A C# compile error	Fix the first Console error, then restart Unity
The old room appears after a rebuild	A hand-saved `Environment.prefab` replaces the generated hall	Delete it before OCS > Build Experience Scene
A red Console message about an older builder	The scene was built by an earlier version of the builder	Rebuild the scene
An `.asset` field silently does nothing	A field-name typo; Unity ignores unknown keys	`tools/headless-tests/run.sh` flags it
Black screen in the headset	OpenXR not enabled under the Android tab	XR Plug-in Management, Android, OpenXR
The sticks and buttons do nothing	No Oculus Touch Controller Profile	Add it under OpenXR interaction profiles
Two editors fight over `ProjectVersion.txt`	Different Unity versions	Everyone uses 6000.6.1f1

Open items

This playbook stays `in progress` until these are closed.

- **Run it on a Quest 2.** Record the frame rate through the power-on wave and while tokens stream, plus three cold starts, as a new capture in `evidence/captures/`. That run would close milestones V1 and V2.
- **Outside-viewer test (V6).** Three people not on the team should be able to explain afterwards what the rigs do and where the model runs. If they can't, rewrite the narration and test again.
- **Power-on audio cue (V4).** Not added yet.
- **Real art (V7).** Cable routing, baked normal maps and texture atlases are not done.
- **The VR shows the target path, not the live one.** Since 2026-10-15 the live system differs from the VR in these ways:

VR SHOWS	LIVE, AS OF THE SPRINT 4 PLAYBOOK
Broker with Redis and RDS	EC2 Nginx → admission gateway → orchestrator → llama.cpp worker. No broker, Redis or RDS
`rig-2` , Qwen3.8 27B on GPUs 0–3 (4 cards, an assumption in `gpu_count`)	Rig 2 serves `qwen3.6:35b-a3b` on all 8 GPUs, through Open WebUI only

VR SHOWS	LIVE, AS OF THE **SPRINT 4 PLAYBOOK**
About 12 tokens/s (deck figure)	Rig 2 about 27 tok/s on the MoE model. Rig 1 Worker A about 8.7 tok/s
Eight EVGA GTX 1070 SC cards	Rig 2's GPU 0 is an 11 GB 1080 Ti (`TENSOR_SPLIT=11,8,...`)

These are data (`sample-trace.json` , `SystemGraph.asset` , `RigFacts.asset`), not code. The real fix is a recorded trace: once the gateway or orchestrator logs the fields in the VR's metrics contract, a captured request can replace the hand-written one.

- **Live mode (V8, optional).** Not started, by design. It needs a read-only trace endpoint.

Sources

- `VirtualLLMVisualizer` , branch `claude/nifty-cannon-b34hkf` , commits `5fec212` (2026-09-16) to `92544c4` (2026-10-12)
- In that repo: `docs/experience-design.md` , `docs/milestones.md` , `docs/demo-checklist.md` , `docs/setup-runbook.md` , `docs/metrics-contract.md`
- Headless run of `tools/headless-tests/run.sh` on `92544c4` , 2026-10-14 (output above)
- Inference on both rigs, for the live state of Rig 2